Data Programming with R - Assignment 2

Charalampos Sarakatsanis | 25267220

Task 1: Manipulation

Question 1.

Load the dataset cycle_25.csv, save it as a tibble. What is the size (number of rows and columns) of this dataset?

```
#Importing the tidyverse library.
library(tidyverse)
```

```
#Loading the dataset as a tibble object.
cycle = as_tibble(read.csv("cycle_25.csv"))
#Number of rows and columns
dim(cycle)
```

```
[1] 5832   27
```

So the number of rows is 5832 and the number of columns is 27.

Question 2

Use the function skim_without_charts() from the package skimr on this dataset, and briefly explain in your own words what the function is doing.

```
#Importing skimr library
library(skimr)
#Using skim_without_charts() on cycle_25 dataset.
skim_without_charts(cycle)
```

Table 1: Data summary

Name	cycle
Number of rows	5832
Number of columns	27
Column type frequency:	
character	1
numeric	26
Group variables	None

Variable type: character

skim_variable	n_missing	complete_rate	min	max	empty	n_unique	whitespace
Time	0	1	19	21	0	5831	0

Variable type: numeric

skim_variable	n_missing	complete_rate	mean	sd	p0	p25	p50	p75	p100
Clonskeagh.Totem	1	1.00	58.43	63.06	0	11.0	43	74.0	423
Clonskeagh.Totem.Cyclist.North	1	1.00	28.05	38.81	0	5.0	20	34.0	334
Clonskeagh.Totem.Cyclist.South	1	1.00	30.38	37.64	0	6.0	19	38.0	262
Dun.Laoghaire...Peoples.Park.Cyclist	1	1.00	37.97	42.60	0	2.0	25	60.0	314
Dun.Laoghaire...Peoples.Park.Cyclist.North	1	1.00	19.31	22.60	0	1.0	12	29.0	166
Dun.Laoghaire...Peoples.Park.Cyclist.South	1	1.00	18.66	21.97	0	1.0	11	30.0	157
Dun.Laoghaire...Seapoint.Beach.Cyclist	25	1.00	46.26	52.50	0	3.0	31	72.0	450
Dun.Laoghaire...Seapoint.Beach.South.Cyclist	25	1.00	24.10	29.51	0	2.0	14	37.0	274
Dun.Laoghaire...Seapoint.Beach.North.Cyclist	25	1.00	22.17	25.88	0	1.5	14	34.0	232
Dun.Laoghaire...York.Road.Cyclist	49	0.99	34.90	38.39	0	3.0	23	56.0	263

skim_variable	n_missing	complete_rate	mean	sd	p0	p25	p50	p75	p100
Dun.Laoghaire...York.Road.Cyclist.North	49	0.99	19.06	22.61	0	1.0	11	30.0	162
Dun.Laoghaire...York.Road.Cyclist.South	49	0.99	15.84	17.69	0	1.0	10	25.0	140
Glenageary.Cyclist	1	1.00	27.68	27.49	0	6.0	20	42.0	214
Glenageary.North.Cyclist	1	1.00	14.13	13.92	0	3.0	10	21.0	106
Glenageary.South.Cyclist	1	1.00	13.55	13.93	0	2.0	10	20.0	108
Grange.Road.Totem	164	0.97	9.89	10.17	0	1.0	7	15.0	65
N11.Before.UCD	1	1.00	18.27	26.96	0	3.0	12	21.0	227
N11.Before.UCD.North.Cyclist	1	1.00	16.76	26.43	0	2.0	10	19.0	222
N11.Before.UCD.South.Cyclist	1	1.00	1.51	2.23	0	0.0	1	2.0	17
N11.Totem.Inbound	1	1.00	15.41	24.38	0	2.0	9	17.0	211
Rock.Road.Park	1	1.00	23.29	26.67	0	1.0	14	36.0	175
Rock.Road.Park.IN	1	1.00	12.87	16.55	0	1.0	7	18.5	123
Rock.Road.Park.OUT	1	1.00	10.42	13.07	0	0.0	6	16.0	91
Rock.Road.Totem.Inbound	1	1.00	26.60	39.18	0	4.0	17	32.0	350
Wyattville.Rd...Bicycles.From.N11	1	1.00	2.77	3.12	0	0.0	2	4.0	21
Wyattville.Rd...Bicycles.Towards.N11.Cyclist	1	1.00	2.84	3.21	0	0.0	2	4.0	24

Using the function skim_without_charts() from the package skimr, summarizes and returns key statistics and completeness information for each variable and is an alternative to the summary() command. To be more precise:

- For each category/column that contains numerical data, skim_without_charts displays: the number of missing values, the percentage of which the dataset is complete, the mean, the standard deviation, the 0th percentile, the 25th percentile, the 50th percentile, the 75th percentile, and the 100th percentile.
- In case the column contains characters, skim_without_charts will give us: Number of missing values, complete rate of the column, minimum number of characters in an entry of the column, maximum number of characters, number of empty values, number of unique entries and number of whitespaces.

Skimr commands used to be combined with other tidyverse functions and can be used to get the summary of specific columns also. The difference between skim() and skim_without_charts() is that skim() displays a full summary with charts (small histograms) and skim_without_charts displays the same summary without the charts. We usually use it when we want a simpler and more clean output or if we want to export the results of that command.

Question 3

Write some code to check that the variable Time is stored using an appropriate class for a date, and the other variables are numeric, fix them if they aren't.

```
#For each columns we select its name if does not contain numerical data.
cycle |> select(where (~ !is.numeric(.))) |> names()
```

```
[1] "Time"
```

The only column that does not contain numerical data is the column Time, since that was the only one printed by the code.

```
#Now lets check if Time has date type of data correctly stored in it.
#First we import lubridate.
library(lubridate)
class(cycle$Time)
```

```
[1] "character"
```

Now that we know that the class of cycle `Time` is not Date, we have to change that

```
#We are changing cycle['Time'] with the date data type version of it.
#mdy_hm, means month - dat - year - time (Hour:minutes), just
#like cycle['Time'] is consisted
cycle = cycle |>
  mutate(Time = mdy_hm(Time))
#Now let's check the class again
class(cycle$Time)
```

```
[1] "POSIXct" "POSIXt"
```

POSIXct indicates we have a `date` data type variable. To be more precise it is the date month day, year and time (hour-minutes). Now all of our valuables are numeric apart from Time who is POSIXct.

Question 4

Write some code to check that each date has 24 hourly entries, and that there are 243 different dates (i.e. from January 1st, 2025 until August 31st, 2025).

```
no24entries = FALSE #After the for, we will use this value in order
#to print whether
#all days have 24 hourly entries or not
for (i in unique(yday(cycle$Time))) #for every unique day
{
  cycle2 = cycle |> filter(yday(Time) == i) #We can not use the argument
#"cycle |> filter(yday(Time) == i" inside
  #nrow, so we created cycle2

  if (nrow(cycle2) != 24) #if every day does not
    #have 24 time entries
  {
    no24entries = TRUE #Then we will get into the if after the for
    #and we will print: There are days existing who do not
    #have 24 hourly entries in them
  }
}

if (no24entries)
{
  print('There are days existing who do not have 24 hourly entries in them')
}else
{
  print('All days have 24 hourly entries')
}
```

```
[1] "All days have 24 hourly entries"
```

```
if (min(as_date(cycle$Time)) == as_date('2025-01-01')
    #if the earliest date cycle has is 2025-01-01.
    && (max(as_date(cycle$Time)) == as_date('2025-08-31'))
    #if the latest date cycle has is 2025-08-31.
    && (length(unique(as_date(cycle$Time))) == 243))
  #if we have 243 different days.
{
  cat('In our dataset, there are 243 different
      dates from January 1st, 2025 until August 31st, 2025')
```

```
}else
{
  print('In our dataset, there are not 243 different
        dates from January 1st, 2025 until August 31st, 2025')
}
```

```
In our dataset, there are 243 different
      dates from January 1st, 2025 until August 31st, 2025
```

After running our code, we have concluded that all days have 24 hourly entries and that In our dataset, there are 243 different dates from January 1st, 2025 until August 31st, 2025.

Question 5

Remove any columns with at least 2 missing data points.

```
#From cycle we select to print the names of the columns who have at least 2
#missing data points.
#We did that, to be informed about the columns that are about to be removed.
print('The following columns where removed: ')
```

```
[1] "The following columns where removed: "
```

```
names(cycle |> select(where(~ sum(is.na(.)) >= 2)))
```

```
[1] "Dun.Laoghaire...Seapoint.Beach.Cyclist"
[2] "Dun.Laoghaire...Seapoint.Beach.South.Cyclist"
[3] "Dun.Laoghaire...Seapoint.Beach.North.Cyclist"
[4] "Dun.Laoghaire...York.Road.Cyclist"
[5] "Dun.Laoghaire...York.Road.Cyclist.North"
[6] "Dun.Laoghaire...York.Road.Cyclist.South"
[7] "Grange.Road.Totem"
```

```
#From cycle we select to keep the columns who have less
#than 2 missing data points. (The other columns are removed).
cycle = cycle |> select(where(~ sum(is.na(.)) < 2))
```

Question 6

Notice that all the remaining columns with the exception of "Time" contain exactly 1 missing value. Investigate the reason for this: write a check to find the date and write a brief explaination.

```
for (i in names(cycle)){
  for (j in 1:nrow(cycle)){
    if (is.na(cycle[j, i])){
      cat(names(cycle[j, i]), 'has a null value at',
          as.character(cycle$Time[j]), "\n \n")
    }
  }
}
```

```
Clonskeagh.Totem has a null value at 2025-03-30 02:00:00

Clonskeagh.Totem.Cyclist.North has a null value at 2025-03-30 02:00:00

Clonskeagh.Totem.Cyclist.South has a null value at 2025-03-30 02:00:00

Dun.Laoghaire...Peoples.Park.Cyclist has a null value at 2025-03-30 02:00:00

Dun.Laoghaire...Peoples.Park.Cyclist.North has a null value at 2025-03-
30 02:00:00

Dun.Laoghaire...Peoples.Park.Cyclist.South has a null value at 2025-03-
30 02:00:00

Glenageary.Cyclist has a null value at 2025-03-30 02:00:00

Glenageary.North.Cyclist has a null value at 2025-03-30 02:00:00

Glenageary.South.Cyclist has a null value at 2025-03-30 02:00:00

N11.Before.UCD has a null value at 2025-03-30 02:00:00

N11.Before.UCD.North.Cyclist has a null value at 2025-03-30 02:00:00

N11.Before.UCD.South.Cyclist has a null value at 2025-03-30 02:00:00

N11.Totem.Inbound has a null value at 2025-03-30 02:00:00
```

```
Rock.Road.Park has a null value at 2025-03-30 02:00:00

Rock.Road.Park.IN has a null value at 2025-03-30 02:00:00

Rock.Road.Park.OUT has a null value at 2025-03-30 02:00:00

Rock.Road.Totem.Inbound has a null value at 2025-03-30 02:00:00

Wyattville.Rd...Bicycles.From.N11 has a null value at 2025-03-30 02:00:00

Wyattville.Rd...Bicycles.Towards.N11.Cyclist has a null value at 2025-03-
30 02:00:00
```

After running the code chunk above, we notice that each variable has exactly one missing value, and this occurs (for all variables) at the same time-stamp: `2025-03-30 02:00`. This is an expected issue, because on this date and time the clocks change to **summer time**, which means that the hour 02:00 is skipped. However, the database still has this time-stamp pre-recorded. This indicates that the data is collected and then matched to the `Time` variable. So when the clock jumps forward, the 02:00 slot exists even though no sensor data were logged for that hour, since it never existed. All other entries that remain in our dataset, for all columns are complete with no null data.

```r
#Since we do not need the null value entry, becuase
#it is a time that is not supposed to exist lets remove it.
cycle = cycle |> na.omit()
```

Question 7

Most locations have bidirectional data, but some columns contain the overall location value (the sum of the two directions), which is redundant. Use regular expressions to help you identify the columns with directional data and remove the columns with the overall location value.[Make sure to keep the column with "Time"]

```r
names(cycle)
```

```
 [1] "Time"
 [2] "Clonskeagh.Totem"
 [3] "Clonskeagh.Totem.Cyclist.North"
 [4] "Clonskeagh.Totem.Cyclist.South"
```

```
 [5] "Dun.Laoghaire...Peoples.Park.Cyclist"
 [6] "Dun.Laoghaire...Peoples.Park.Cyclist.North"
 [7] "Dun.Laoghaire...Peoples.Park.Cyclist.South"
 [8] "Glenageary.Cyclist"
 [9] "Glenageary.North.Cyclist"
[10] "Glenageary.South.Cyclist"
[11] "N11.Before.UCD"
[12] "N11.Before.UCD.North.Cyclist"
[13] "N11.Before.UCD.South.Cyclist"
[14] "N11.Totem.Inbound"
[15] "Rock.Road.Park"
[16] "Rock.Road.Park.IN"
[17] "Rock.Road.Park.OUT"
[18] "Rock.Road.Totem.Inbound"
[19] "Wyattville.Rd...Bicycles.From.N11"
[20] "Wyattville.Rd...Bicycles.Towards.N11.Cyclist"
```

```r
library(stringr)

#Defining the directions that exist in the naming pattern.
directions = "(North|South|IN|OUT|Inbound|Outbound)"

#Keeping the names of the directional columns and the Time column.
directioncols = names(cycle)[
  # Keeping the Time column.
  names(cycle) == "Time" |
  #Keeping the columns that contain the directions.
  str_detect(names(cycle), directions)
]

#Selecting the columns we want to keep.
cycle = cycle |>
  select(all_of(directioncols))

names(cycle)
```

```
 [1] "Time"
 [2] "Clonskeagh.Totem.Cyclist.North"
 [3] "Clonskeagh.Totem.Cyclist.South"
 [4] "Dun.Laoghaire...Peoples.Park.Cyclist.North"
 [5] "Dun.Laoghaire...Peoples.Park.Cyclist.South"
 [6] "Glenageary.North.Cyclist"
```

```
 [7] "Glenageary.South.Cyclist"
 [8] "N11.Before.UCD.North.Cyclist"
 [9] "N11.Before.UCD.South.Cyclist"
[10] "N11.Totem.Inbound"
[11] "Rock.Road.Park.IN"
[12] "Rock.Road.Park.OUT"
[13] "Rock.Road.Totem.Inbound"
```

Question 8

Add two columns: one containing the name of the day of the week and the other the month. Check that these two columns are ordered factors, fix them if they aren't.

```
#Creating a new column that based on the variable time,
#each day will recorded by it's name and not as a number.
cycle["Day_name"] = weekdays(cycle$Time)
#Creating a new column that based on the variable Time,
#each month will be recorded by its name and not as a number.
cycle["Month_name"] = month(cycle$Time, label = TRUE, abbr = FALSE)
#Let's check if the columns are ordered factors.
class(cycle$Day_name)
```

```
[1] "character"
```

```
class(cycle$Month_name)
```

```
[1] "ordered" "factor"
```

So, `Name_day` is not an ordered factor, Let's change that.

```
#We will use factor(), in order to turn our column into a factor.
cycle$Day_name = factor(cycle$Day_name,levels =  c("Monday",
                                                    "Tuesday", "Wednesday",
                                                    "Thursday", "Friday",
                                                    "Saturday", "Sunday"),
                        ordered = TRUE)
class(cycle$Day_name)
```

```
[1] "ordered" "factor"
```

Now, both columns that we used, are factors.

Question 9

Use dplyr::relocate() to put the new columns with the month and day of the week as the second and third columns of the dataset. Print the column names to confirm that the code worked properly

```
#We import dplyr
library(dplyr)
#The first column is Time, so we will place our variables after that.
cycle = cycle |> relocate(Month_name, Day_name, .after = Time)
#Checking if relocate worked properly.
names(cycle)
```

```
 [1] "Time"
 [2] "Month_name"
 [3] "Day_name"
 [4] "Clonskeagh.Totem.Cyclist.North"
 [5] "Clonskeagh.Totem.Cyclist.South"
 [6] "Dun.Laoghaire...Peoples.Park.Cyclist.North"
 [7] "Dun.Laoghaire...Peoples.Park.Cyclist.South"
 [8] "Glenageary.North.Cyclist"
 [9] "Glenageary.South.Cyclist"
[10] "N11.Before.UCD.North.Cyclist"
[11] "N11.Before.UCD.South.Cyclist"
[12] "N11.Totem.Inbound"
[13] "Rock.Road.Park.IN"
[14] "Rock.Road.Park.OUT"
[15] "Rock.Road.Totem.Inbound"
```

After looking at the results, the code worked properly.

Task 2 Analysis

Question 1

Use functions from base R to compute which months had the highest and the lowest overall bike traffic across all locations (i.e. total bike traffic in all location in the whole month)

```r
#We will need to find the max sum of all columns per month apart from Time,
#Day_name and Month_name.
month_sum = c()
for (i in unique(cycle$Month_name)){

  month_sum[i] = 0
  current_month = cycle |>
    filter(Month_name == i) |> select(-Time, -Month_name, -Day_name)
  month_sum[i] <- sum(replace(current_month, is.na(current_month), 0))
}
#Printing the month with the highest overall bike traffic
cat(names(month_sum[month_sum == max(month_sum)]),
    "has the highest overall bike traffic, with a total of:",
    max(month_sum), "\n")
```

```
May has the highest overall bike traffic, with a total of: 189620
```

```r
#Printing the month with the lowest overall bike traffic
cat(names(month_sum[month_sum == min(month_sum)]),

    "has the lowest bike traffic, with a total of:",

    min(month_sum))
```

```
January has the lowest bike traffic, with a total of: 107640
```

The month with the highest overall Bike traffic is May and the month with the lowest is January.

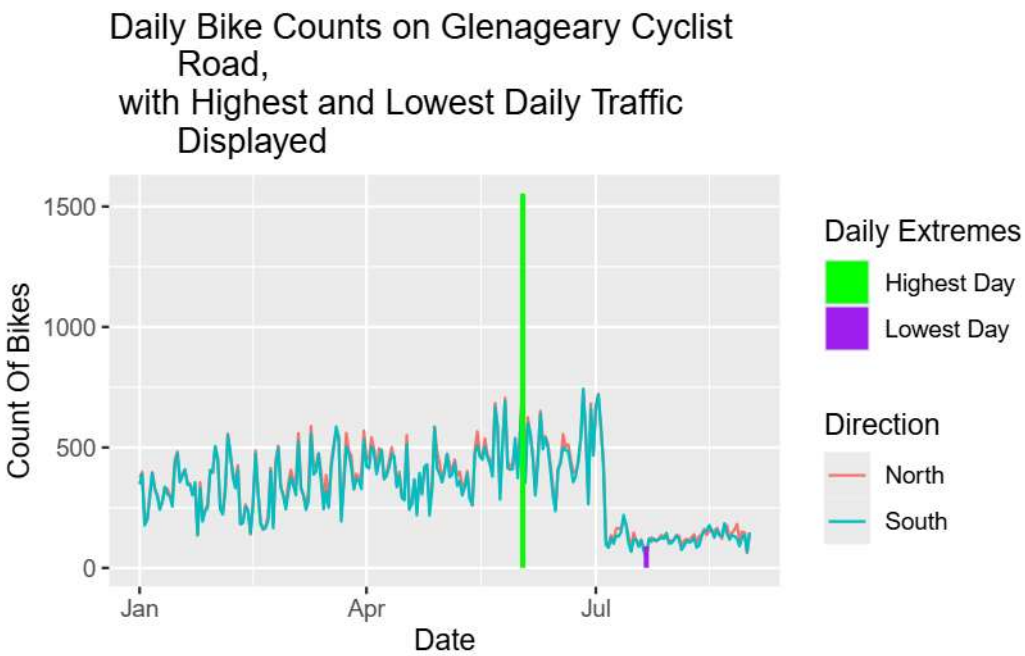
Question 2

Use ggplot2 to create a plot displaying two time series of daily bike traffic in both directions for one location of your choice (i.e. overall daily total for each one of the two directions calculated sepatately). Add two vertical bars to mark the days with the highest and the lowest daily bike passage. [The two time series must be in the same plot]. Outline on your findings.

```r
library(ggplot2)
day_sum_n = c()
day_sum_s = c()
#A variable that contains all the days of the dataset once.
```

```r
daysc = unique(as.Date(cycle$Time))

for (i in daysc){
  #computing the sum of each direction for the current day in the for loop
  current_day = cycle |>
    filter(as.Date(Time) == i) |>
    select(Time, Glenageary.North.Cyclist,

         Glenageary.South.Cyclist)
  #Adding that sum to a table for each direction
  day_sum_n[as.character(i)] = sum(current_day$Glenageary.North.Cyclist)
  day_sum_s[as.character(i)] = sum(current_day$Glenageary.South.Cyclist)
}

#Creating the dataset that will be used to plot - it is
#consisted by the unique days, the traffic sum for each
#direction and the total sum in irder to find the most busy day.
plotdf = tibble(date = daysc,
                Ndir = day_sum_n,
                Sdir = day_sum_s,
                day_sum = day_sum_n + day_sum_s)

max_day = plotdf |> filter(day_sum == max(day_sum))
min_day = plotdf |> filter(day_sum == min(day_sum))

#Changing the dataset to a form convenient for plotting.This
#way we will easily add both directions in one plotting and also add colors.
plotdf <- plotdf |> pivot_longer(
      cols = c(Ndir, Sdir),
      names_to = "Direction",
      values_to = "Count"
    )
#Creating a path graph with two bars one for the day with
#the maximum traffic for both directions and one for the day with the minimum.
myplot = ggplot(plotdf, aes(x = date, y = Count, color = Direction))+
  geom_path()+
  scale_color_discrete(labels =
                        c("Ndir" = "North", "Sdir" = "South"))+
  geom_col(data = max_day, aes(x = date, y=day_sum,
                               fill="Highest Day", width = 2),
           inherit.aes = FALSE)+
  geom_col(data = min_day,
```

```
            aes(x = date, y=day_sum,
                fill = "Lowest Day", width = 2),
            inherit.aes = FALSE)+
  scale_fill_manual(name = "Daily Extremes",
                    values = c("Highest Day" = "green",
                               "Lowest Day" = "purple")) +
  labs(x = "Date", y = "Count Of Bikes",
       title = "Daily Bike Counts on Glenageary Cyclist
       Road, \n with Highest and Lowest Daily Traffic
       Displayed")
myplot
```




```
cat(format(max_day$date, "%Y-%m-%d"), "was the day
    with the most traffic with", max_day$day_sum, "total cyclists \n")
```

```
2025-06-02 was the day
    with the most traffic with 1553 total cyclists
```

```
cat(format(min_day$date, "%Y-%m-%d"),
    "was the day with the least traffic with",
    min_day$day_sum, "total cyclists")
```

```
2025-07-21 was the day with the least traffic with 89 total cyclists
```

Overall the bike traffic in both directions is similar, with both prevailing by a little some days. Based on the day, it is noticeable the fact that the traffic changes drastically, with many local peaks. The highest traffic is from March to July. From July to January, the lowest traffic period is noticeable. 2025-06-02 was the day with the most traffic with 1553 total cyclists and it is located within the peak traffic period. The day with the lowest traffic is 2025-07-21 with 89 total cyclists and is located within the least traffic period.

Question 3

We want to analyse the overall bike traffic usage depending on the day of the week, with a particular focus on the morning rash hour (7am-9am) and evening rash hour (5pm-7pm). To do so, create two tables displaying for each day of the week and time slot the median and average overall bike traffic. Outline your findings.

```
cycle = as_tibble(read.csv("cycle_25.csv"))
cycle = na.omit(cycle)
cycle = cycle |>
  mutate(Time = mdy_hm(Time))

#Creating a new column that based on the variable time,
#each day will recorded by it's name and not as a number.
cycle["Day_name"] = weekdays(cycle$Time)
#Creating a new column that based on the variable Time,
#each month will be recorded by its name and not as a number.
cycle["Month_name"] = month(cycle$Time, label = TRUE,
                            abbr = FALSE)

cycle$Day_name = factor(cycle$Day_name,levels =
                          c("Monday", "Tuesday",
                            "Wednesday", "Thursday",
                            "Friday", "Saturday",
                            "Sunday"),
                        ordered = TRUE)

#Computing the total traffic and adding it as a column
cycle["Total"] = cycle$Clonskeagh.Totem +
                  cycle$Glenageary.North.Cyclist +
                  cycle$Glenageary.South.Cyclist +
                  cycle$N11.Before.UCD.North.Cyclist +
```

```
                    cycle$N11.Before.UCD.South.Cyclist +
                    cycle$Wyattville.Rd...Bicycles.From.N11 +
                    cycle$Wyattville.Rd...Bicycles.Towards.N11.Cyclist

#Creating a column with hour.
cycle["Hour"] = hour(cycle$Time)

#computing the total traffic from 07:00 a.m. to 09:00 a.m.
seven_to_nine = cycle |>
  filter(Hour >= 7, Hour < 9) |>
  group_by(Day_name) |>
  summarise(
    wday_mean   = mean(Total),
    wday_median = median(Total),
    .groups = "drop"
  )

#computing the total traffic from 17:00 p.m. to 19:00 p.m.
five_to_seven = cycle |>
  filter(Hour >= 17, Hour < 19) |>
  group_by(Day_name) |>
  summarise(
    wday_mean   = mean(Total),
    wday_median = median(Total),
    .groups = "drop"
  )

#Printing the results
cat("\n7-9 Morning table:\n \n")
```

```
7-9 Morning table:
```

```
print.data.frame(seven_to_nine)
```

```
   Day_name wday_mean wday_median
1    Monday 281.60938       249.0
2   Tuesday 390.12500       311.0
3 Wednesday 380.72059       334.0
4  Thursday 343.98529       298.0
```

```
5    Friday 224.80882       209.0
6  Saturday  55.39706        50.5
7    Sunday  52.22059        51.0
```

```
cat("\n17-19 Evening table:\n \n")
```

```
17-19 Evening table:
```

```
print.data.frame(five_to_seven)
```

```
   Day_name wday_mean wday_median
1    Monday  254.3906       263.5
2   Tuesday  315.3438       316.5
3 Wednesday  305.6765       316.0
4  Thursday  287.2353       290.0
5    Friday  191.0000       194.0
6  Saturday  114.9118       109.5
7    Sunday  106.7794       100.5
```

Task 3 Creativity

Do something interesting with these data! Create one plot and one table showing something we have not discovered above already and outline your findings (the plot and the table must display different findings). [You can use any of the variables from the original dataset cycle_25.csv if you wish to].

First of all, let's find the most popular places for cycling from the original dataset. In order to perform this task, we will use all the variables of the **cycle_25** that sum both directions, and in order to avoid problems with NAs, we will use na.omit(), because changing null values with the mean or using similar techniques is not a good alternative, since as displayed in the diagram of Task 2, second question — the number of people passing by can vary. To prove this, let's summarise our dataset with skim and have a look at the individual standard deviation and percentiles for each place. Afterwards let's do the same for a dataset that has the total number of cyclists that passed by for all the roads.

```
#Reading again the dataset.
cycle = as_tibble(read.csv("cycle_25.csv"))
#Printing summarise details.
skim(cycle)
```

Table 4: Data summary

Name	cycle
Number of rows	5832
Number of columns	27
Column type frequency:	
character	1
numeric	26
Group variables	None

Variable type: character

skim_variable	n_missing	complete_rate	min	max	empty	n_unique	whitespace
Time	0	1	19	21	0	5831	0

Variable type: numeric

skim_variable	n_missing	complete_rate	mean	sd	p0	p25	p50	p75	p100	hist
Clonskeagh.Totem	1	1.00	58.43	63.06	0	11.0	43	74.0	423	
Clonskeagh.Totem.Cyclist.North	1	1.00	28.05	38.81	0	5.0	20	34.0	334	
Clonskeagh.Totem.Cyclist.South	1	1.00	30.38	37.64	0	6.0	19	38.0	262	
Dun.Laoghaire...Peoples.Park.Cyclist	1	1.00	37.97	42.60	0	2.0	25	60.0	314	
Dun.Laoghaire...Peoples.Park.Cyclist.North	1	1.00	19.31	22.60	0	1.0	12	29.0	166	
Dun.Laoghaire...Peoples.Park.Cyclist.South	1	1.00	18.66	21.97	0	1.0	11	30.0	157	
Dun.Laoghaire...Seapoint.Beach.Cyclist	25	1.00	46.26	52.50	0	3.0	31	72.0	450	
Dun.Laoghaire...Seapoint.Beach.South.Cyclist	25	1.00	24.10	29.51	0	2.0	14	37.0	274	
Dun.Laoghaire...Seapoint.Beach.North.Cyclist	25	1.00	22.17	25.88	0	1.5	14	34.0	232	
Dun.Laoghaire...York.Road.Cyclist	49	0.99	34.90	38.39	0	3.0	23	56.0	263	

skim_variable	n_missing	complete_rate	mean	sd	p0	p25	p50	p75	p100	hist
Dun.Laoghaire...York.Road.Cyclist.North	49	0.99	19.06	22.61	0	1.0	11	30.0	162	
Dun.Laoghaire...York.Road.Cyclist.South	49	0.99	15.84	17.69	0	1.0	10	25.0	140	
Glenageary.Cyclist	1	1.00	27.68	27.49	0	6.0	20	42.0	214	
Glenageary.North.Cyclist	1	1.00	14.13	13.92	0	3.0	10	21.0	106	
Glenageary.South.Cyclist	1	1.00	13.55	13.93	0	2.0	10	20.0	108	
Grange.Road.Totem	164	0.97	9.89	10.17	0	1.0	7	15.0	65	
N11.Before.UCD	1	1.00	18.27	26.96	0	3.0	12	21.0	227	
N11.Before.UCD.North.Cyclist	1	1.00	16.76	26.43	0	2.0	10	19.0	222	
N11.Before.UCD.South.Cyclist	1	1.00	1.51	2.23	0	0.0	1	2.0	17	
N11.Totem.Inbound	1	1.00	15.41	24.38	0	2.0	9	17.0	211	
Rock.Road.Park	1	1.00	23.29	26.67	0	1.0	14	36.0	175	
Rock.Road.Park.IN	1	1.00	12.87	16.55	0	1.0	7	18.5	123	
Rock.Road.Park.OUT	1	1.00	10.42	13.07	0	0.0	6	16.0	91	
Rock.Road.Totem.Inbound	1	1.00	26.60	39.18	0	4.0	17	32.0	350	
Wyattville.Rd...Bicycles.From.N11	1	1.00	2.77	3.12	0	0.0	2	4.0	21	
Wyattville.Rd...Bicycles.Towards.N11.Cyclist	1	1.00	2.84	3.21	0	0.0	2	4.0	24	

```
#removing null values for algorithms to run smoothly.
cycle = na.omit(cycle)
```

After looking at the individual standard deviations and percentiles, the results vary and the assumptions based on the Task 2, question 2 plot were right. Overall, for each street the results are unpredictable and especially in columns that have missing values, the standard deviation surpasses even the mean.

So let's remove the null values. The column Grange.Road.Totem has a total of 164 null values, but still it is a place itself and is approximately 97.72% complete, which is enough for it to stay, with those rows removed.

After looking at those results, we can also conclude that since the standard deviations vary as well as the percentiles, in ways that do not align with the mean, even if we use the weekdays as a mean to fill the null values, it will still not work. So the rate of change of traffic for each road is unique and unpredictable.

Let's now check the standard deviation, the mean and the percentiles for all places.

```r
# Checking if the direction is located at the end or followed by anything.
at_end = sub("\\.(North|South|IN|OUT|Inbound|Outbound).*", "", directioncols)

# Checking if the direction is located in the middle, for
#example Glenageary.North.Cyclist.
in_middle = sub("(.*)\\.(North|South)(\\.Cyclist)$", "\\1\\3", directioncols)

# Combining both sets.
sumcols = union(at_end, in_middle)

# Keeping only those that are real columns, not to get errors
sumcols = intersect(sumcols, names(cycle))

#select does not work for time, because it is not applicable
#for class "character"
sumcols2 = setdiff(sumcols, "Time")
#Computing the total traffic for all roads
Totalcyc = cycle |>
  transmute(
    Time,
    Sums = rowSums(across(all_of(sumcols2)))
  )

skim(Totalcyc)
```

Table 7: Data summary

Name	Totalcyc
Number of rows	5620
Number of columns	2
Column type frequency:	
character	1
numeric	1
Group variables	None

Variable type: character

skim_variable	n_missing	complete_rate	min	max	empty	n_unique	whitespace
Time	0	1	19	21	0	5620	0

Variable type: numeric

skim_variable	n_missing	complete_rate	mean	sd	p0	p25	p50	p75	p100	hist
Sums	0	1	268.35	263.96	0	43	211	400	1685	

The standard deviation of the sum of all places is also a number equal to the mean and it varies a lot, since the number of the 50th percentile is close to that of the standard deviation. This means that we should not replace null values even for the total sum of traffic for all places.

Now, let's find the most popular day–time among all.

```
#Printing the date with the highest bike traffic
cat(
  Totalcyc$Time[Totalcyc$Sums == max(Totalcyc$Sums)],
  "is the day with the highest bike traffic with a
  total of:",
  max(Totalcyc$Sums))
```

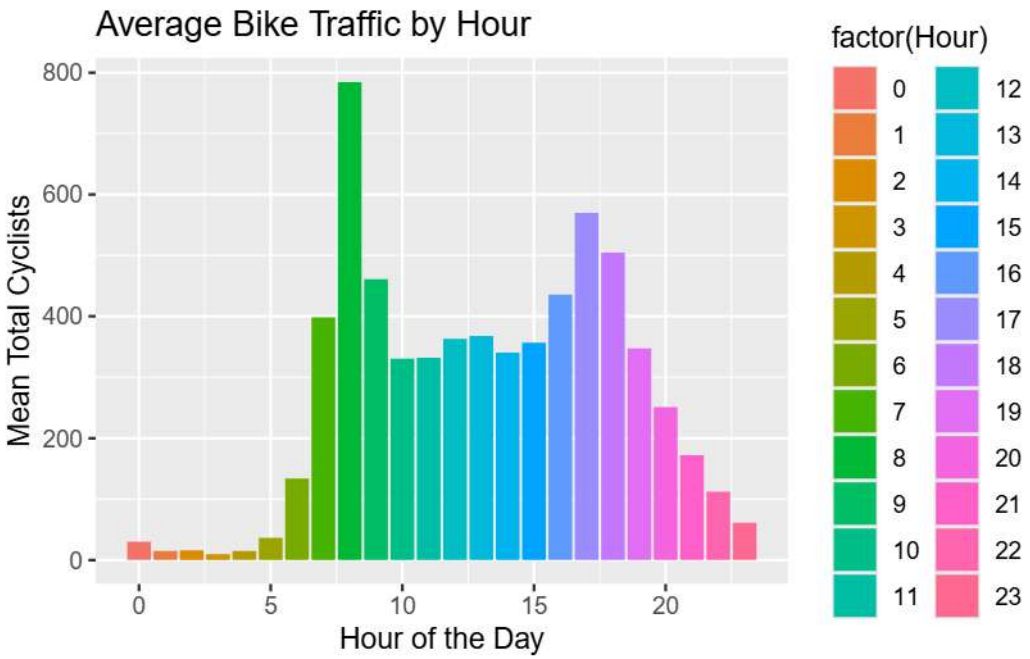
```
May 13, 2025 8:00 AM is the day with the highest bike traffic with a
  total of: 1685
```

The timestamp with the highest recorded cyclist count occurred on May 13 around 09:00. At that time an unusually intense morning peak is detected. Possible explanations might be the weather conditions or local events; a real explanation cannot be confirmed because the dataset contains no information about external factors. Also, the 13th of May is a date that Bike Week is running and might have contributed to the unusually high bike activity detected. However, more specific data have not been given by the dataset to further the analysis on that matter.

We have already done the most popular and unpopular day in Task 2 – Question 2, so a good idea would be to create a boxplot for every single place to really understand how the traffic varies in each place and get a better glimpse of the unpredictability of our dataset.

```
#removing null values for the algorithms to run
cycle = na.omit(cycle)
#Converting time again to POSIXct type
cycle = cycle |> mutate(Time = mdy_hm(Time))
```

```r
#Adding the new column with hour
cycle["Hour"] = hour(cycle$Time)
#adding the column with totals, we have that already
#computed, so we will add that column to cycle.
cycle["Total"] = Totalcyc$Sums

#Creating the right dataset that will be used for the plot.
#The cycle grouped by hour with the mean of total traffic for each hour.
hourplot = cycle |> group_by(Hour) |>
  summarise(Mean_Traffic = mean(Total,
                                na.rm = TRUE),
            .groups = "drop")

#Creating the bar plot
myplot = ggplot(hourplot,
                aes(x = Hour,
                    y = Mean_Traffic,
                    fill = factor(Hour))) +
  geom_col() +
  labs(
    title = "Average Bike Traffic by Hour",
    x = "Hour of the Day",
    y = "Mean Total Cyclists"
  )


myplot
```



For the bar chart, we notice that the peak hours are the ones we expected and mentioned in previous questions.

- From 07:00 until 09:00 is the morning peak time. At 08:00 the highest by far traffic level occurs.
- The afternoon peak hours occur between 16:00 and 19:00. Apart from the peak hour during the afternoon at 17:00, which is considerably lower than the morning's, all other peak hour values seem very similar.
- From 20:00 until 07:00, the bicycle traffic is very low. From the peak hours that the plot displays, it is safe to conclude that bike traffic peaks during the hours that people go to work and leave work.

An interesting fact is that we have an "in-between" quiet peak from 12:00 to 13:00 that may reflect lunchtime breaks.